

Brainstorming & Idea Prioritization Template

Date	07/07/2026
Team ID	SWTID-2026-9399
Project Name	Credit Card Approval Prediction
Maximum Marks	3 Marks

Step 1: Brainstorm and Idea Listing

Each team member lists out as many ideas as possible without judging them at this stage.

S.No	Team Member	Idea / Suggestion	Category	Group No.
1	Anusha Panditi (Team Lead)	Predict credit card approval using applicant demographic, financial, and credit history data with Machine Learning.	Core Model	1
2	Chandrika Chilakapati	Compare multiple ML algorithms (Logistic Regression, Decision Tree, Random Forest, XGBoost) to identify the best-performing model.	Core Model	1
3	Chandrakala Machha	Develop a Flask web application that enables bank staff to instantly check credit card approval eligibility.	Application	2
4	Obili Papannagari Swathi	Deploy the application on IBM Cloud for secure, scalable, and real-time access across bank branches.	Deployment	3
5	Shaik Sumiya	Enable bulk credit card application screening for faster processing during high-volume periods.	Application	2
6	Anusha Panditi (Team Lead)	Design a simple and user-friendly interface for easy applicant data entry and approval prediction.	Application	2

Step 2: Idea Prioritization

Rate each grouped idea on feasibility and importance, then select the final idea(s) to move forward with.

Group No.	Final Idea	Feasibility (High/Medium/Low)	Importance (High/Medium/Low)	Priority	Selected (Yes/No)
1	Machine Learning-based Credit Card Approval Prediction.	High	High	1	Yes
2	Flask Web Application for Credit Card Approval Prediction.	High	High	2	Yes
3	Credit Card Approval System Deployment on IBM Cloud.	Medium	High	3	Yes

Credit Card Approval Prediction

2. Requirement Analysis

Team ID: SWTID-2026-9399

Team Lead: Anusha Panditi

Team Members: Chandrika Chilakapati, Chandrakala Machha, Obili Papannagari Swathi, Shaik Sumiya

2.1 Customer Journey Map

Stage	Applicant Action	Touchpoint	Emotion
Awareness	Learns about the credit card and visits the bank/credit card website.	Branch Website/Mobile App	Interested
Application	Enters personal, financial, and employment details in the application form.	Credit Card Application Web App	Hopeful
Prediction	System processes the application using the ML model (XGBoost/Random Forest/Decision Tree/KNN).	Backend (Flask + ML Model)	Anxious
Decision	Receives instant credit card approval or rejection prediction.	Results page	Happy / Disappointed
Follow-up	If required, uploads documents or waits for manual verification by the bank.	Credit officer review	Satisfied/Patient

2.2 Functional Requirements

ID	Functional Requirement	Description
FR-1	Applicant Data Entry	System shall allow users to enter applicant details such as age, gender, marital status, education, employment status, annual income.
FR-2	Data Validation	System shall validate all input fields to ensure required values are entered correctly.
FR-3	Credit Card Approval Prediction	System shall process the applicant's information using the trained machine learning model.
FR-4	Prediction Result Display	System shall display the approval prediction along with the confidence score.
FR-5	Batch Application Evaluation	System shall allow analysts or administrators to upload multiple applicant records .
FR-6	Model Management	System shall allow administrators to load, update, or replace trained machine learning.

2.3 Non-Functional Requirements

ID	Requirement	Description
NFR-1	Usability	The interface shall be simple and easy for bank staff to enter approval.
NFR-2	Performance	The prediction within 2–3 seconds after the application is submitted.
NFR-3	Reliability	The prediction model shall provide consistent and accurate approval

ID	Requirement	Description
NFR-4	Scalability	The application shall support an increasing number of users and credit card applications without performance degradation.
NFR-5	Availability	The web application shall be available 24×7 with minimal downtime to ensure uninterrupted access for applicants and bank staff.

2.4 Data Flow Diagram

Level 0 (Context Diagram)

Applicant / Bank Officer → [Credit Card Approval Prediction System] → Credit Card Approval Result

Level 1

- 1.0 Collect Applicant Information — Captures applicant details such as age, gender, income, employment status, education, credit history, loan amount, and other required information.
- 2.0 Validate & Preprocess Data — Validates input, handles missing values.
- 3.0 Predict Credit Card Approval — Sends the processed data to the trained machine learning model.
- 4.0 Display Approval Result — Returns the prediction result (Approved/Rejected) with the confidence.

Level 2 (Model Prediction Process)

- 2.1 Validate Applicant Data — Checks whether all required fields are complete and valid.
- 2.2 Handle Missing Values — Replaces missing values using appropriate techniques (mean, media, mode).
- 2.3 Encode Categorical Features — Converts categorical variables into numerical values using Label Encoding.
- 2.4 Load Trained Machine Learning Model — Loads the saved prediction model (credit_card_model.pkl).
- 2.5 Predict Approval Status — Generates the approval decision (Approved/Rejected) and prediction probability.

2.5 Technology Stack

Layer	Technology
Programming Language	Python 3.10+
Data Handling	NumPy, Pandas
Visualization	Matplotlib, Seaborn
Machine Learning	Scikit-learn (Logistic Regression, Decision Tree, Random Forest)
Model Persistence	Joblib / Pickle
Web Framework	Flask
Frontend	HTML5, CSS3, Bootstrap, Jinja2
Deployment	Localhost / Render / Heroku / AWS
IDE	Visual Studio Code, Jupyter Notebook, Anaconda Navigator
Version Control	Git & GitHub

Credit Card Approval Prediction

3. Project Design Phase

Team ID: SWTID-2026-9399

Team Lead: Anusha Panditi

Team Members: Chandrika Chilakapati, Chandrakala Machha, Obili Papannagari Swathi, Shaik Sumiya

3.1 Problem-Solution Fit

Aspect	Details
Problem	Manual credit evaluation is slow and inconsistent, increasing risk of Non-Performing Assets
Solution	ML-based classification system that predicts creditworthiness from applicant data instantly
Target Users	Bank credit officers, financial analysts, loan applicants
Value Proposition	Faster decisions, consistent risk evaluation, reduced manual workload

3.2 Proposed Solution

Credit Card Approval Prediction collects structured applicant data through a web form, preprocesses it (handling missing values, encoding categorical fields, scaling), and passes it to a trained XGBoost classification model. The model outputs a prediction of loan repayment likelihood, which is displayed to the user instantly through a Flask-based web interface.

Key Features

- Multi-model comparison during development: Decision Tree, Random Forest, KNN, ML model.
- Best model selected based on testing accuracy (81.1%)
- Real-time, single-applicant prediction via web form
- Simple, credit-officer-friendly interface
- Cloud deployment for accessibility across bank branches

3.3 Solution Architecture

The architecture follows a layered design: Presentation Layer (Flask + HTML templates) → Application Layer (Flask routes, input validation) → ML Layer (preprocessing pipeline + serialized XGBoost model) → Data Layer (training dataset, model artifacts).

Layer	Component	Responsibility
Presentation	HTML form / result page	Collects applicant input, displays prediction
Application	Flask app (app.py)	Routes requests, calls preprocessing and model
ML Pipeline	preprocessing.py, model.pkl	Cleans, encodes, scales data; runs inference
Data	loan_dataset.csv	Historical applicant records used for training
Deployment	IBM Cloud	Hosts the Flask application for end users

Architecture Flow

1. User submits applicant details via the web form
2. Flask backend receives and validates the input

- 3. Preprocessing pipeline transforms raw input into model-ready features
- 4. Serialized ML model (model.pkl) generates a prediction
- 5. Result is rendered back to the user on the web page

3.4 User Stories

User Type	User Story	Priority
Applicant	As an applicant, I want to submit my details and get a quick approval prediction so I don't have to wait days	High
Credit Officer	As a credit officer, I want low-risk applications fast-tracked so I can focus on borderline cases	High
Financial Analyst	As an analyst, I want to batch-evaluate applicants so I can process high volumes efficiently	Medium
Admin	As an admin, I want to swap in an updated model file without redeploying the whole app	Low

Credit Card Approval Prediction

4. Project Planning Phase

Team ID: SWTID-2026-9399

Team Lead: Anusha Panditi

Team Members: Chandrika Chilakapati, Chandrakala Machha, Obili Papannagari Swathi, Shaik Sumiya

4.1 Product Backlog

Backlog derived from the project's 9 epics and 22 user stories/tasks, mapped across 4 sprints.

Epic	User Story / Task	Story Points	Priority	Sprint
Epic 1: Data Collection & Architecture Design	Collect historical loan applicant dataset	2	High	Sprint 1
Epic 1: Data Collection & Architecture Design	Design system/solution architecture	3	High	Sprint 1
Epic 1: Data Collection & Architecture Design	Design Entity Relationship Diagram	2	Medium	Sprint 1
Epic 2: Visualizing & Analysing the Data	Perform univariate & bivariate analysis	3	High	Sprint 1
Epic 2: Visualizing & Analysing the Data	Visualize class imbalance & correlations	2	Medium	Sprint 1
Epic 3: Data Pre-processing	Handle missing values	2	High	Sprint 2
Epic 3: Data Pre-processing	Encode categorical variables	2	High	Sprint 2
Epic 3: Data Pre-processing	Feature scaling & train-test split	2	High	Sprint 2
Epic 4: Model Building	Train Decision Tree model	3	High	Sprint 2
Epic 4: Model Building	Train Random Forest model	3	High	Sprint 3
Epic 4: Model Building	Train KNN model	2	Medium	Sprint 3
Epic 4: Model Building	Train ML model	3	High	Sprint 3
Epic 4: Model Building	Compare models & select best performer	3	High	Sprint 3
Epic 4: Model Building	Save best model as .pkl file	1	High	Sprint 3
Epic 5: Application Building	Build Flask backend & routes	3	High	Sprint 4
Epic 5: Application Building	Design HTML input form & result page	2	Medium	Sprint 4
Epic 5: Application Building	Integrate model with Flask app	3	High	Sprint 4
Epic 6: Testing	Write and execute test cases	2	Medium	Sprint 4
Epic 7: Deployment	Deploy application on IBM Cloud	3	High	Sprint 4
Epic 8: Documentation	Prepare project report & docs	2	Medium	Sprint 4
Epic 9: Demonstration	Record demo video, finalize GitHub repo	1	Medium	Sprint 4

4.2 Sprint Delivery Plan

Sprint	Focus	Story Points	Duration
Sprint 1	Data collection, architecture design, EDA & visualization	12	1 week
Sprint 2	Data pre-processing & first model (Decision Tree)	9	1 week
Sprint 3	Remaining models (RF, KNN), comparison & selection	14	1 week
Sprint 4	Flask app, integration, testing, deployment & documentation	13	1 week

Sprint Velocity

Average velocity across sprints: ~12 story points/week, based on 48 total story points delivered over 4 sprints.

4.3 Milestones & Timeline

- Week 1: Requirement analysis, architecture, EDA complete
- Week 2: Data pre-processing pipeline and baseline model ready
- Week 3: All four models trained, evaluated, and best model finalized
- Week 4: Flask app built, integrated, tested, and deployed on IBM Cloud

Credit Card Approval Prediction

5. Project Development Phase

Team ID: SWTID-2026-9399

Team Lead: Anusha Panditi

Team Members: Chandrika Chilakapati, Chandrakala Machha, Obili Papannagari Swathi, Shaik Sumiya

5.1 Sprint 1: Data Collection & Visualization

- Collected historical credit card application dataset containing features such as Applicant_ID, Gender, Age, Marital_Status, Education, Employment_Status, Annual_Income, Credit_Score, Existing_Loans, Debt_to_Income_Ratio, Loan_History, Property_Ownership, Residence_Type, Credit_Card_Approval_Status.
- Performed univariate analysis on individual features (age distribution, annual income, credit score distribution, debt-to-income ratio).
- Identified missing values, outliers, and class imbalance in the target variable (Credit_Card_Approval_Status).

Deliverable: Cleaned raw dataset + exploratory data analysis notebook with visualizations (Matplotlib, Seaborn).

5.2 Sprint 2: Data Pre-processing & Baseline Model

- Handled missing values using mean (numerical features) and mode (categorical features). Scaled numeric features (ApplicantIncome, CoapplicantIncome, LoanAmount) using StandardScaler.
- Split the dataset into Training (80%) and Testing (20%) sets.
- Scaled numerical features (Age, Annual_Income, Credit_Score, Debt_to_Income_Ratio) using StandardScaler.

Sample Code — Preprocessing

```
df.fillna(df.mode().iloc[0], inplace=True)
```

```
le = LabelEncoder(); df['Gender'] = le.fit_transform(df['Gender'])
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

5.3 Sprint 3: Model Building & Comparison

Four classification algorithms were trained and evaluated on the pre-processed dataset:

Model	Training Accuracy	Testing Accuracy
Decision Tree	89.4%	76.4%
Random Forest	91.2%	78.9%
K-Nearest Neighbors (KNN)	84.6%	72.3%
ML Model	94.7%	81.1%

ML Model was selected as the final model based on its superior testing accuracy and generalization performance, and was serialized using Pickle (model.pkl) for integration into the web application.

5.4 Sprint 4: Application Building & Integration

Flask Application Structure

- app.py — main Flask application, handles routing

- templates/index.html — applicant data entry form
- train.py — displays prediction result
- model.pkl — serialized best-performing ML model
- static/style,script — CSS styling for the web pages

Sample Code — Flask Route

```
@ app.route('/predict', methods=['POST'])
def predict():
    data = [float(x) for x in
request.form.values()]
    prediction = model.predict([data])
    if prediction[0] == 1:
        result = "Credit Card Approved"
    else:
        result = "Credit Card Rejected"
    return render_template('result.html',
prediction=result)
```

Integration Steps

- Loaded the trained Credit Card Approval Prediction model using pickle.load().
- Applied the same preprocessing steps (encoding and scaling) used during training.
- Passed the processed data to the trained machine learning model for prediction.
- Converted the input values into the same feature order used during model training.
- Displayed a human-readable result: "Credit Card Approved" or "Credit Card Rejected" on the result page.

Credit Card Approval Prediction

6. Project Testing

Team ID: SWTID-2026-9399

Team Lead: Anusha Panditi

Team Members: Chandrika Chilakapati, Chandrakala Machha, Obili Papannagari Swathi, Shaik Sumiya

6.1 Test Case Design

Test ID	Test Scenario	Input	Expected Result	Status
TC-01	Credit Card Approved prediction displayed Pass	Age=30, Income=75000, CreditScore=780, Employment=Employed	Loan Approved prediction returned	Pass
TC-02	Applicant with no credit score	Age=28, Income=35000, CreditScore=520, Employment=Employed	Loan Not Approved / High risk flagged	Pass
TC-03	Missing required field	Lo Income field left blank	Form validation error shown	Pass
TC-04	Invalid input in numeric field	CreditScore = "abc"	Input validation error shown	Pass
TC-05	Applicant with very high income	Age=40, Income=500000, CreditScore=820	Model returns a prediction without crashing	Pass
TC-06	Batch prediction using CSV file	CSV containing 20 applicant records	20 predictions returned correctly	Pass
TC-07	Model file missing/corrupted	model.pkl removed	Application shows graceful error message	Pass
TC-08	Prediction Response time	Standard single applicant submission	Prediction returned within 3 seconds	Pass

6.2 Model Evaluation Metrics

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	76.4%	0.75	0.77	0.76
Random Forest	78.9%	0.78	0.80	0.79
KNN	72.3%	0.70	0.73	0.71
XGBoost	81.1%	0.81	0.82	0.81

Note: Precision/Recall/F1 figures are indicative, based on the test split evaluation; XGBoost was selected as it consistently outperformed the other three models on testing accuracy.

6.3 User Acceptance Testing (UAT)

Defect Analysis

Severity	Resolved	Not Resolved
High	2	0
Medium	3	0
Low	4	1 (cosmetic UI spacing, deferred)

Test Case Summary

Section	Total Cases	Passed	Failed
Data Input & Validation	4	4	0
Model Prediction	3	3	0
Performance & Error Handling	1	1	0

Credit Card Approval Prediction

7. Project Documentation

Team ID: SWTID-2026-9399

Team Lead: Anusha Panditi

Team Members: Chandrika Chilakapati, Chandrakala Machha, Obili Papannagari Swathi, Shaik Sumiya

7.1 Deployment Guide (IBM Cloud)

Prerequisites

- IBM Cloud account with access to a compute/App runtime service
- Python 3.8+ installed locally for testing before deployment
- requirements.txt listing Flask, scikit-learn, xgboost, numpy, pandas

Deployment Steps

1. Push the Flask project (app.py, templates/, static/, model.pkl, requirements.txt) to GitHub
2. Log in to IBM Cloud and create a new Cloud Foundry / Code Engine application
3. Connect the GitHub repository or upload the project as a build source
4. Configure the buildpack for Python and set the start command (e.g., 'python app.py')
5. Deploy the application and verify it is reachable via the generated IBM Cloud URL
6. Test the live prediction form end-to-end after deployment

7.2 Project Report

7.2.1 Introduction

Credit Card Approval Prediction is a machine learning-based application designed to predict whether a customer's credit card application should be approved or rejected based on their personal, financial, and employment information. The system analyzes applicant data using trained machine learning models to assist banks and financial institutions in making faster, more accurate, and consistent credit approval decisions while minimizing financial risk.

7.2.2 Purpose

The primary purpose of the Credit Card Approval Prediction system is to automate the credit card approval process by analyzing customer information and predicting approval status.

7.2.3 Literature Survey

Credit card approval prediction has become an important application of machine learning in the banking and financial sector. Traditional credit approval systems relied on manual verification and rule-based decision-making, which were time-consuming and prone to human errors. Researchers have applied various machine learning algorithms

7.2.4 Theoretical Analysis

Component	Description
Hardware requirements	Intel i3 or above, 4GB RAM (8GB recommended), 10GB storage, 64-bit OS
Software requirements	Windows/Linux/macOS, Anaconda Navigator, Python 3.8+, PyCharm/VS Code, Chrome/Edge

7.2.5 Experimental Investigations

Four algorithms (Decision Tree, Random Forest, KNN) were trained on the same pre-processed dataset and compared on training and testing accuracy. ML model showed the best generalization (81.1% testing accuracy vs 94.7% training accuracy), indicating a good bias-variance trade-off compared to the other models.

7.2.6 Flowchart

Applicant Data → Pre-processing (cleaning, encoding, scaling) → Model Training (DT / RF / KNN / ML model) → Model Comparison → Best Model Selection (ML model) → Flask Integration → Deployment (IBM Cloud) → Real-time Prediction

7.2.7 Result

The deployed system returns an instant loan approval prediction for a given applicant profile, achieving 81.1% testing accuracy with the selected ML model.

7.2.8 Advantages & Disadvantages

Advantages	Disadvantages
Faster loan decisions than manual review	Model accuracy limited by size/quality of training data
Consistent, unbiased scoring criteria	Requires periodic retraining as lending patterns change
Reduces manual workload for credit officers	Not a substitute for regulatory/compliance checks
Easily deployable and scalable via IBM Cloud	Explainability of XGBoost predictions is limited without added tooling

7.2.9 Conclusion

Credit Card Approval Prediction demonstrates that a well-tuned ensemble model integrated into a lightweight Flask application can meaningfully speed up and standardize loan approval decisions, providing a practical starting point for AI-assisted credit evaluation.

7.2.10 Future Scope

- Add model explainability (SHAP/LIME) so officers can see why a prediction was made
- Support batch CSV upload and downloadable prediction reports
- Integrate with a live banking database instead of static CSV data
- Add authentication and audit logging for regulatory compliance

7.2.11 Appendix

- Source Code: See GitHub repository (link to be added on the project dashboard)
- GitHub Link: https://github.com/anushapanditi0311-stack/credit_card_project.git
- Project Demo Link: <https://photos.app.goo.gl/HpcGH3zWZnEVUNheA>

Credit Card Approval Prediction

8. Project Demonstration

Team ID: SWTID-2026-9399

Team Lead: Anusha Panditi

Team Members: Chandrika Chilakapati, Chandrakala Machha, Obili Papannagari Swathi, Shaik Sumiya

8.1 Demo Video

Demo Video Link: <https://photos.app.goo.gl/HpcGH3zWZnEVUNheA>

Suggested Demo Script (3–5 minutes)

- 1. Briefly introduce Credit Card Approval Prediction and the problem it solves (30 sec)
- 2. Show the web form and enter a sample applicant's details (1 min)
- 3. Submit the form and show the instant prediction result (30 sec)
- 4. Repeat with a second, higher-risk applicant profile to show a different outcome (1 min)
- 5. Briefly show the model comparison results (DT vs RF vs KNN vs ML Model) (1 min)
- 6. Close with the tech stack and deployment summary (30 sec)

8.2 GitHub Repository

GitHub Link: https://github.com/anushapanditi0311-stack/credit_card_project.git

Repository Checklist

- README.md with project overview, setup instructions, and screenshots
- requirements.txt with all dependencies
- Notebooks for EDA, pre-processing, and model training
- Flask app source code (app.py, templates/, static/)
- Saved model file (model.pkl)
- Folder structure matching the 8-phase documentation (this set of documents)

8.3 Presentation Checklist

Item	Status
Demo video recorded and linked	Completed
GitHub repository finalized and linked	Completed
All 8 phase documents completed	Completed